

Project Proposal: Temporal Circuit Stability for LLM Reliability

Andrei Kopanev, and Jay Himanshu Jivandas, and Jayden Vinson

The Ohio State University

CSE 5525: Foundations of Speech and Language Processing

1 Main Goals

The central scientific question is whether mechanistic explanations based on attribution graphs exhibit meaningful temporal organization during multi-step generation, and whether that temporal organization can be turned into an interpretable verification signal.

Prior work suggests that attribution-style graphs can make LLM behavior more legible, but different lines of work emphasize different aspects of that structure. In mechanistic circuit tracing and the “biology” framing (Ameisen et al., 2025; Lindsey et al., 2025), graphs are primarily a tool for isolating the internal mechanisms that implement a behavior. Other approaches use graphs as summaries of generation: GraphGhost (Dai et al., 2026) extracts local signal-propagation graphs for next-token prediction and then aggregates them across samples to identify stable, global patterns, while CRV (Zhao et al., 2025) treats chain-of-thought computational graphs as execution traces and learns structural cues that predict step correctness (with targeted interventions as validation). Finally, CAGE (Walker and Ewetz, 2025) builds token-level attribution graphs over prompts and generations to track inter-token influence and improve faithfulness of context attributions. Despite this progress, it remains unclear whether the *within-completion* evolution of mechanistic structure during generation carries a distinct, usable signal about reasoning reliability.

In this project, we explicitly model the temporal dimension of mechanistic graphs during multi-step generation (especially chain-of-thought) and test whether temporal circuit stability provides an interpretable verification signal.

1.1 Primary hypotheses

- H1: Temporal structure exists. For a fixed prompt and a single generated completion, the per-token (i.e., per decoding-step) attribution/circuit graphs

$G_1, \dots, G_T$ are not arbitrary; they contain recurring substructures (“stable cores”) and systematic changes (“churn”) across consecutive decoding steps.

- H2: Temporal circuit stability helps predict correctness. On average, correct solutions exhibit higher temporal consistency (e.g., larger stable cores, higher step-to-step overlap, lower churn) than incorrect solutions, but stable-yet-wrong (“confidently wrong”) failures can occur. We hypothesize that (i) stability features provide predictive signal beyond simple confounds (sequence length, entropy, average log-probability), and (ii) combining stability with complementary black-box signals (e.g., entropy/log-probability or self-consistency) yields higher verifier performance than either family of features alone.
- H3: Prompt-conditioned invariants exist across runs. Across multiple sampled continuations from the same prompt, there is a measurable “prompt-conditioned core” (graph substructure common across runs). The strength/variance of this core correlates with reliability: prompts with unstable or highly variable cores are more likely to produce inconsistent or incorrect reasoning.

1.2 Importance

Reliability and faithfulness are increasingly central concerns for LLM deployment. Existing verification often remains black-box (output-based) or gray-box (activation-based) and can struggle to explain why failures occur. A temporally grounded, mechanistic signal could improve verification while remaining interpretable and potentially actionable via targeted interventions.

1.3 Novelty

GraphGhost and CRV already operate in multi-token settings, but they do not make temporal evolution itself the primary object of quantitative study. GraphGhost aggregates across examples to find globally stable propagation patterns; CRV focuses

on predicting step correctness from graph structure (and interventions), without explicitly formalizing "temporal stability trajectories" within a completion or "prompt-conditioned invariants" across multiple runs as the main signal.

1.4 Secondary / stretch goals

- **Mechanistic case studies:** Identify and visualize a few recurring temporal motifs associated with specific failure modes (e.g., arithmetic slips, distractor reliance), and test whether they are task/domain-specific (connecting to CRV’s "domain specificity" observation).
- **Causal validation:** Use stability/motif findings to select intervention targets (e.g., features frequently appearing in transient "detour" motifs correlated with failure) and test whether interventions reduce errors, following CRV’s causal direction but guided by temporal metrics.

2 NLP task

Given a natural-language reasoning prompt (e.g., math/logic word problem), the model generates a multi-token solution (optionally chain-of-thought) and a final answer. We extract per-token attribution/circuit graphs during generation and study how temporal graph structure differs between correct vs. incorrect outputs.

- **Input example:** "If a store has 12 boxes with 24 candies each and sells 35 candies, how many remain?"
- **Output example:** "Total = $12 \times 24 = 288$; remaining = $288 - 35 = 253$. Final answer: 253."
- **Evaluation label:** ground-truth final answer (exact match / numeric equality).

3 Datasets

We will use a subset of the GSM8K dataset for multi-step mathematical reasoning, since it provides naturally long, multi-token solutions and a clean final-answer label suitable for automatic correctness evaluation. The dataset contains 7,473 training examples and 1,319 evaluation examples (Cobbe et al., 2021).

We sample 300 questions from the GSM8K train split and divide them by question into 240 train and 60 dev. We separately sample 50 questions from the GSM8K test split as the final held-out test set.

All splits are performed by question (not by completion), so no prompt appears in more than one split, preventing prompt-specific memorization.

Preprocessing will be minimal. We will (1) format each item into a consistent prompt template (e.g., "Question: ... Answer:" and optionally require a standardized "Final answer:" line), (2) parse the ground-truth numeric answer from the dataset’s provided answer field, and (3) extract and normalize the model’s predicted final numeric answer using a robust rule (e.g., a regular expression over the last stated number or the value after "Final answer:"), including normalization of commas and whitespace for comparison.

For each question, we generate K independent completions ($K = 1-3$ under compute constraints; higher K if feasible). Each completion is labeled correct/incorrect by exact-match numeric answer parsing.

4 Neural Methods

We will use a pretrained, open-weight instruction-tuned language model and pretrained interpretability artifacts, and then implement our temporal analysis on top of the resulting attribution graphs.

Model + interpretability features. We will run experiments on the instruction-tuned Gemma 3 1B-IT model (Google, 2025a) and use pretrained cross-layer MLP transcoders from the Gemma Scope 2 release. In particular, we plan to use the Gemma Scope 2 "transcoder-262k" (Google, 2025b) configuration (width 262k) for Gemma-3-1B-IT.

Attribution graph construction. We will use Anthropic’s circuit-tracer library (Anthropic Safety Research, 2025) to generate per-token attribution graphs during autoregressive generation, using the pretrained (cross-layer) transcoders as the feature basis. This yields a sparse computational graph describing how active features and errors contribute to downstream features and output logits at each token generation step.

Our method. Our main contribution is a temporal analysis layer on top of these per-token graphs. For each completion we extract a sequence of attribution graphs $G_1, \dots, G_T$ (one per generated token) and compute temporal-structure metrics that quantify how "stable" the underlying mechanistic computation is across decoding.

Graph representation and preprocessing. We treat each step graph as a directed, weighted multi-graph $G_t = (V_t, E_t, w_t)$ where nodes represent active features (e.g., transcoder features at specific

layers/positions) and edges represent traced contributions between features, each with a signed attribution weight (we use $|w_t(e)|$ for mass-based comparisons). To reduce noise and make metrics comparable across steps, we sparsify graphs via *cumulative-mass thresholding* (used in both Anthropic’s circuit-tracer work and CRV (Zhao et al., 2025)): sort edges in decreasing order of $|w_t(e)|$ and retain the minimal prefix S_t such that

$$\frac{\sum_{e \in S_t} |w_t(e)|}{\sum_{e \in E_t} |w_t(e)|} \geq \alpha,$$

where $\alpha \in (0, 1]$ (e.g. $\alpha = 0.95$) is a mass-coverage hyperparameter. We then renormalize so $\sum_{e \in S_t} |w_t(e)| = 1$. As a robustness check, we will also report a hard-threshold ($|w_t(e)| \geq \tau$) variant as an ablation. We compute metrics both on edges and on nodes (where node mass is the sum of incident $|w_t(e)|$).

Temporal stability metrics. For each completion, we compute:

- **Step-to-step overlap / churn:** how similar the attribution graph is between consecutive decoding steps (high overlap / low churn indicates a more consistent circuit across time).
- **Stable-core recurrence:** the size/mass of a “stable core” of edges that persist over many steps, and how often this core changes identity.
- **Persistence of recurring motifs:** the lifetimes and recurrence of small, high-mass local subgraphs (motifs) that reappear across multiple steps.

Metric details. Formal definitions and formulas for overlap/churn and stable cores are provided in Appendix A.

Cross-run prompt-conditioned invariants (H3). For each prompt, we sample R independent completions and compute a run-level core for each run (e.g., edges present in at least a p fraction of decoding steps). We then define the *prompt-conditioned core* as the intersection (or, more generally, the high-frequency subgraph) across runs. We quantify its strength by core size/mass and its variability by across-run overlap; these cross-run stability features are used as additional inputs to the verifier.

Lightweight Verifier We will train a lightweight verifier (e.g., logistic regression, Gradient Boosting Classifier, or a small MLP) on the temporal (and cross-run) stability features to predict completion correctness. The verifier is trained only

on completions from the 240 training questions; model/threshold selection uses the 60-question dev set; final performance is reported on the 50 held-out GSM8K test questions.

Compute feasibility. Per-token graph extraction can be expensive, so we will (i) begin with small-scale sanity checks (≈ 5 – 10 prompts) to validate extraction correctness, and (ii) if needed, subsample timesteps (e.g., last N reasoning tokens, or every m -th token) or limit extraction to fixed windows to control runtime. These fallbacks preserve the core temporal metrics while reducing compute.

Deliverables. We prioritize the temporal overlap/churn + stable-core features and a lightweight verifier; motif-related analyses and causal interventions are considered stretch goals (Section 1.4).

5 Baselines

We compare temporal-graph features against baselines computed on the exact same model generations to ensure apples-to-apples evaluation.

1. **Baseline A (black-box confidence):** log-probability / average token logprob; predictive entropy; optionally self-consistency (agreement across K samples).
2. **Baseline B (static graph):** aggregate graph features ignoring time (graph size/density, node/edge counts, simple structural statistics).
3. **Baseline C (structural/CRV-style, if feasible):** non-temporal circuit/graph summary features that are straightforward to implement (in the spirit of CRV) (Zhao et al., 2025).

Comparison to published results. We will report the model’s standard GSM8K accuracy (published or reproduced) for context, but we do not rely on external published verifier metrics for direct comparison; instead, we report comparative performance across our baselines under the same experimental setup.

6 Evaluation

Our evaluation is scoped to prioritize a minimal, decisive test of whether temporal circuit-stability features add predictive value beyond standard black-box confidence signals.

Minimum evaluation (required). We treat correctness prediction as a binary classification problem (correct vs. incorrect final answer) and report completion-level AUROC and AUPRC. We will

compare three verifiers trained and evaluated on the same generated completions:

- **Black-box only:** confidence features such as average token log-probability and predictive entropy (and self-consistency if available).
- **Temporal only:** temporal overlap/churn, stable-core, and (when used) cross-run stability features.
- **Combined:** black-box + temporal features.

Hyperparameters (including the mass-coverage fraction α) are chosen using the dev questions only. The test set is used once for final reporting.

Optional evaluation extensions. If time/compute permits, we will extend the evaluation with:

- **Prompt-level reporting:** aggregate completion scores per question (e.g., mean score across K completions) to reduce skew from questions with many samples.
- **Uncertainty estimates:** bootstrap confidence intervals over test questions for key metrics (AU-ROC/AUPRC).
- **Failure analysis:** a deeper taxonomy of “stable-yet-wrong” completions and which complementary signals help separate them from correct outputs.
- **Qualitative case studies:** visualizations of stable-core versus transient subgraphs across generation steps.

Task accuracy (context). We will also report the base model’s GSM8K exact-match accuracy to contextualize the distribution of correct vs. incorrect generations used for verifier training and evaluation.

7 Ethical Considerations and Risks

Our project analyzes internal attribution/circuit graphs to study whether temporal structure correlates with correctness. A primary risk is misinterpretation: mechanistic explanations and stability metrics can be mistaken for guarantees of correctness or “certified faithfulness,” especially if results are presented without clear caveats. This could lead to overconfidence in downstream deployments (e.g., using a stability score as a safety filter in high-stakes settings) even though attribution methods are approximate and may not generalize across models, prompts, or domains. To mitigate this, we will frame our findings as empirical correlations and diagnostics rather than proofs, report failure cases and limitations explicitly, and avoid claims

that the method provides definitive guarantees of truthfulness.

A second risk is that interpretability tooling can be dual-use: insights about reliable vs. unreliable internal signatures might be adapted to evade detection (e.g., producing outputs that mimic “stable” signatures without being correct), or could be used to optimize persuasion rather than accuracy. While our work is limited to open models and a benign benchmark (GSM8K) and does not involve user data, we will minimize dual-use concerns by emphasizing transparency about evaluation conditions, not releasing any “hardcoded” detectors as safety claims, and documenting the scope under which our metrics appear to work. Finally, we will ensure reproducibility and responsible reporting by describing all preprocessing, thresholds, and evaluation choices, and by avoiding cherry-picked examples in qualitative analysis.

8 Acknowledgement

We used AI tools in a limited and documented way to support the writing process. The core research idea and project direction were conceived without the use of AI. Subsequently, we used ChatGPT to help refine the methodological framing (e.g., clarifying evaluation design and candidate metrics) and to suggest relevant recent literature to review, which we then independently screened and incorporated as appropriate.

During drafting and editing, portions of the prose were written in close collaboration with AI using Prism (OpenAI’s $\text{\LaTeX}$ editor), primarily for rephrasing, tightening, and improving clarity and organization. All content was ultimately reviewed by the authors, with final wording, factual claims, and citations checked by a human to ensure correctness and alignment with the intended scope of the proposal.

9 Proposed Plan and Team Responsibilities

The project will be implemented in modular stages, with shared design decisions and regular coordination meetings.

Andrei Kopanav will lead overall project design and be primarily responsible for integration of interpretability tooling (circuit-tracer and pretrained transcoders), construction of per-token attribution graphs, and implementation of temporal stability metrics (overlap, churn, stable cores, and cross-run invariants).

Jay Himanshu Jivandas will lead the data and generation pipeline, including GSM8K preprocessing, prompt formatting, model inference, and implementation of baseline confidence and static graph features. He will also assist with training and evaluating the lightweight verifier.

Jayden Vinson will focus on correctness parsing, experiment management, visualization of temporal graph behavior, and support evaluation and analysis of results. He will also contribute to literature review synthesis and final report preparation.

All team members will contribute to experimental design discussions, result interpretation, and writing.

References

- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, and 8 others. 2025. [Circuit tracing: Revealing computational graphs in language models](#). Transformer Circuits.
- Anthropic Safety Research. 2025. [Circuit tracer: Attribution graph extraction for transformer models \(github repository\)](#). <https://github.com/safety-research/circuit-tracer>. Accessed: 2026-02-02.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training Verifiers to Solve Math Word Problems](#). *arXiv preprint*. ArXiv:2110.14168 [cs].
- Xinnan Dai, Xianxuan Long, Chung-Hsiang Lo, Kai Guo, Shenglai Zeng, Dongsheng Luo, and Jiliang Tang. 2026. [GraphGhost: Tracing structures behind large language models](#). *Preprint*, arxiv:2510.08613 [cs].
- Google. 2025a. Gemma 3: 1b it model (hugging face repository). <https://huggingface.co/google/gemma-3-1b-it>. Accessed: 2026-02-02.
- Google. 2025b. Gemma scope 2: 1b it model cross-layer transcoder artifacts (hugging face repository). <https://huggingface.co/google/gemma-scope-2-1b-it/tree/main/clt>. Accessed: 2026-02-02.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, and 8 others. 2025. [On the biology of a large language model](#). Transformer Circuits.
- Chase Walker and Rickard Ewetz. 2025. [Explaining the reasoning of large language models using attribution graphs](#). *Preprint*, arxiv:2512.15663 [cs].
- Zheng Zhao, Yeskendir Koishikenov, Xianjun Yang, Naila Murray, and Nicola Cancedda. 2025. [Verifying chain-of-thought reasoning via its computational graph](#). *Preprint*, arxiv:2510.09312 [cs].

A Metric Details

Step-to-step overlap / churn. Let S_t be the set of retained edges at step t , and let $m_t(e)$ be the normalized edge mass at step t (0 if absent). We compute an unweighted overlap $\text{Jacc}(t) = \frac{|S_t \cap S_{t-1}|}{|S_t \cup S_{t-1}|}$ and a weighted Jaccard overlap

$$\text{wJacc}(t) = \frac{\sum_e \min(m_t(e), m_{t-1}(e))}{\sum_e \max(m_t(e), m_{t-1}(e))}.$$

We define *churn* as $\text{churn}(t) = 1 - \text{wJacc}(t)$.

Stable-core recurrence. For a window size W and persistence threshold $p \in (0, 1]$,

$$C_{t,W,p} = \{e : \frac{1}{W} \sum_{i=t-W+1}^t \mathbb{I}[e \in S_i] \geq p\}.$$

We summarize core size/mass, the fraction of mass captured by $C_{t,W,p}$, and how often the core changes (e.g., via overlap between consecutive cores).